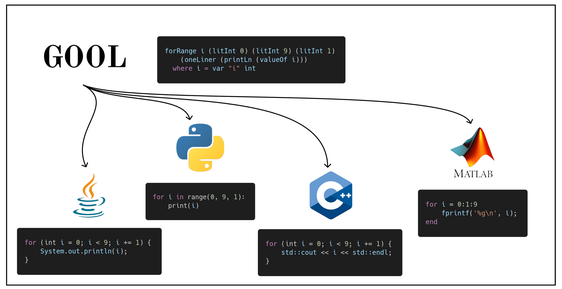
A Generic Representation of Program Organization

Brandon Bosman, Dr. Jacques Carette
Department of Computing and Software, McMaster University

McMaster University | Computing & Software

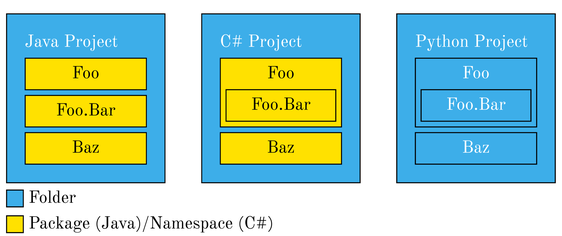
What Is GOOL? [1]

- **G**eneric **O**bject-**O**riented **L**anguage
- "Write once, generate everywhere"



Program Organization

- Different languages have different systems for organizing program logic
- These systems also have different visibility rules



Problem: current GOOL doesn't know about these differences!

- Only knows how to generate code fitting the lowest common denominator of these systems
- This can cause it to generate ugly or broken code for some languages

Generically Representing Program Organization in GOOL

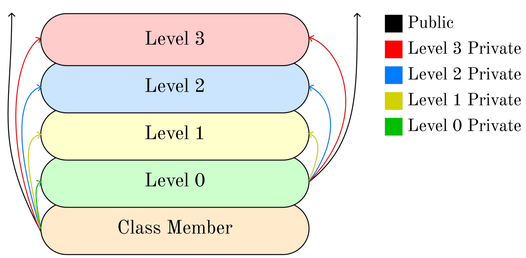


"Package", "module", "namespace" etc. are over-used, so instead we use indexed levels of abstraction.

	GOOL	Java	Swift	MATLAB
Level 3		Project	Module	Project
Level 2		Package	Folder	Namespace
Level 1		File with a static class	File	Folder
Level 0		Static Method	Function	File with a Function

Visibility Options

- Organizational levels form a tree
- Give options for each level to be visible up to any of its enclosing levels
- Some visibilities will not be expressible in some languages
  - Choose the closest visibility that preserves semantics
  - If no visibilities preserve semantics, throw an error for that language



(Arrows from Levels 1-3 left out for readability)

Remaining Questions

- Awkward transition from "Level 3" (abstract name) to "Class Member" (concrete name)
- How to guarantee preservation of semantics and intent?
  - Will require research into scope graphs [2] and module calculi [3]
- Control of organization: GOOL vs. the programmer
  - GOOL should be able to move things around so that they make sense for a given target language
  - The programmer should be able to make overrides about how things should be laid out
  - What if these are in conflict?
- How much work should GOOL do to ensure output is idiomatic?
  - E.g. strictly enforcing Java's one-class-per-file guideline

References

- [1] Carette, J., MacLachlan, B., Smith, S. (2020). GOOL: A Generic Object-Oriented Language. PEPM '20. https://dl.acm.org/doi/10.1145/3372884.3373159
- [2] Zwaan, A., van Antwerpen, H.. (2023). Scope Graphs: The Story so Far. EVCS 2023. https://drops.dagstuhl.de/entities/document/10.4230/OASIcs.EVCS.2023.32
- [3] Bergel, A., Ducasse, S., Nierstrasz, O. (2005). Analyzing Module Diversity. Journal of Universal Computer Science. https://inria.hal.science/hal-01352809v1/file/jucs_11_10_1613_1644_bergel.pdf

Acknowledgements

- Thank you to Dr. Carette for being my supervisor, and to Dr. Smith for co-supervising the Drasil project!
- Thank you to my fellow grad students and summer students who have helped me think through my work!
- Thank you to OGS and NSERC for helping fund my masters work!